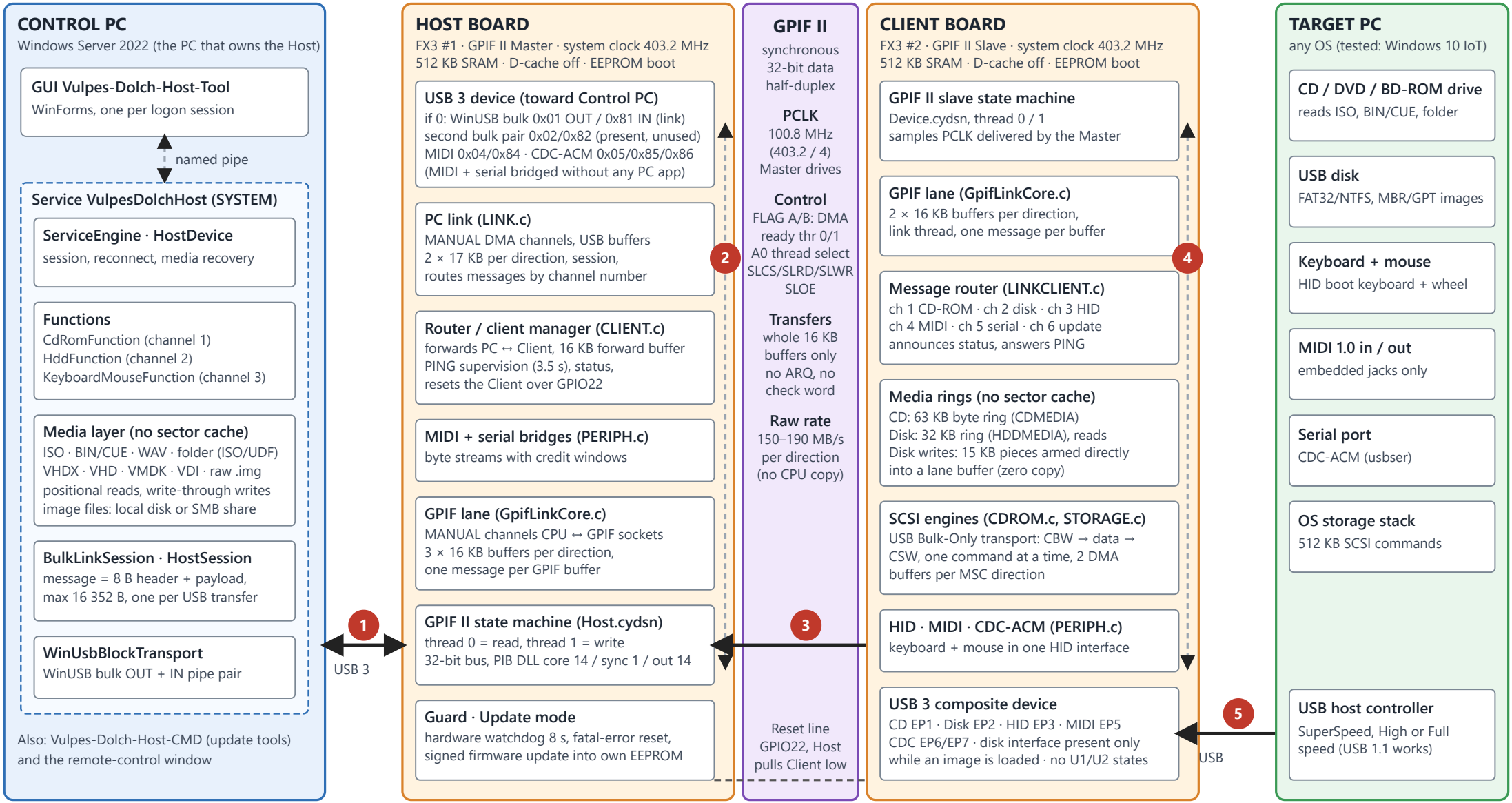
# Vulpes Dolch USB Multidevice – System architecture

Two Cypress EZ-USB FX3 boards (CYUSB3KIT-003) joined by a 32-bit GPIF II bus · firmware Host 1.0.2610.2928 / Client 1.0.2610.3002 · 2026-11-03




**1 Control PC ↔ Host**
USB 3 WinUSB, bulk 1024 B packets; one message per transfer, ≤ 16 352 B. Measured 62 MB/s in / 83 MB/s out (Host-CPU-bound); 188 MB/s w/o CPU.

**2 Host CPU**
Routing + copy between USB buffers and GPIF lane buffers (cache off), ~0.18 ms per 16 KB message; worker threads run below the watchdog guard.

**3 GPIF II bus**
Whole 16 KB buffers, one message each; short buffers wait for the next one. Raw: 156 MB/s Host→Client, 140 Client→Host; no data error in ~61 GB soak.

**4 Client CPU**
Reads: lane buffer copied into ring, ring to USB IN. Writes: USB OUT received into the lane buffer. One SCSI command at a time; ~1.9 ms fixed cost per command.

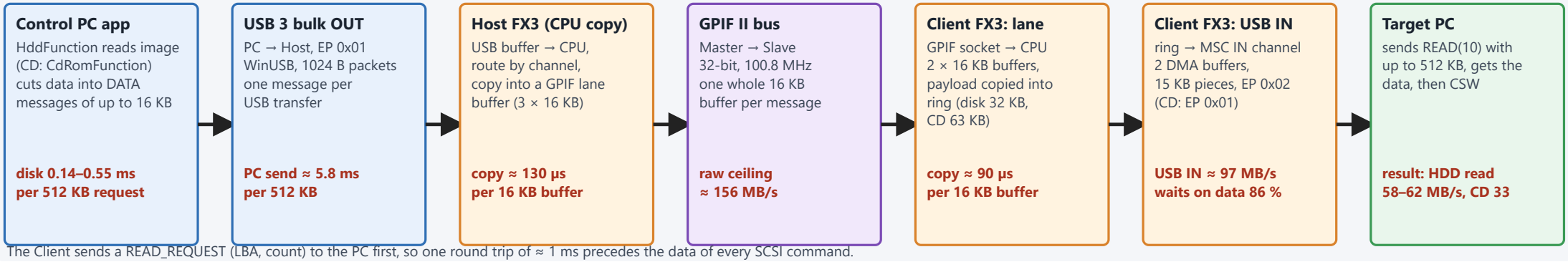
**5 Client ↔ Target PC**
USB 3 / 2 / 1.1 composite device. Windows polls with 512 KB commands; Bulk-Only allows queue depth 1 only. Link power states U1/U2 refused.

Solid arrows: data and control traffic. Dashed grey: internal data path inside a board. Dashed black: Client reset line (Host GPIO22 → Client reset pad).

# Storage data path – copy stages and measured rates

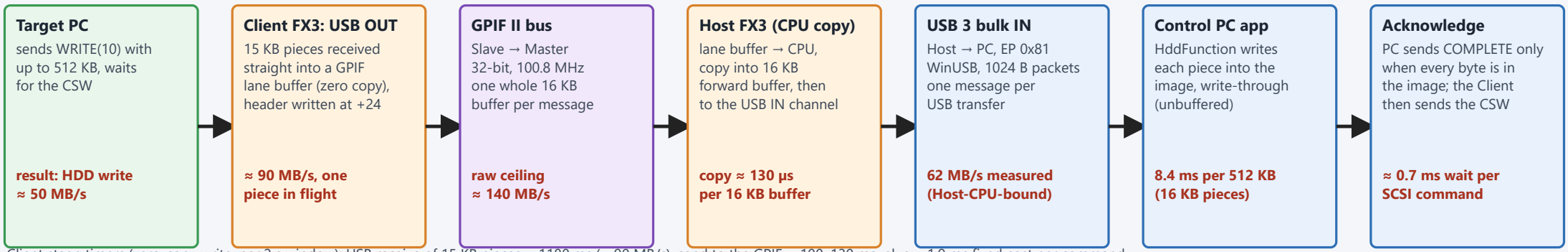
Read (image → Target PC) and write (Target PC → image) through Control PC, Host board, GPIF II and Client board · colour = where the stage runs (blue PC, orange FX3, purple GPIF, green Target)

## READ image file → Target PC (data flows left to right)




The Client sends a READ_REQUEST (LBA, count) to the PC first, so one round trip of ≈ 1 ms precedes the data of every SCSI command.

## WRITE Target PC → image file (data flows left to right)




Client stage timers (zero-copy write, per 2 s window): USB receive of 15 KB pieces ≈ 1180 ms (≈ 90 MB/s), send to the GPIF ≈ 100–130 ms, plus ≈ 1.9 ms fixed cost per command.

### Measured end to end

CrystalDiskMark, 256 MiB, raw FAT32 image, write-through on the Control PC:

**SEQ 1M read 58–62 MB/s · write ≈ 50 MB/s**
**RND 4K read 2.8 · write 2.7 MB/s (QD 1)**
**CD-ROM read ≈ 33 MB/s (64 KB commands)**

Queue depth has no effect (USB Bulk-Only carries one command at a time).

Random 4K: ≈ 1.4 ms per command, of which ≈ 0.9 ms is pure transport latency (Client → GPIF → Host → USB → PC and back); the PC itself needs only 0.2–0.25 ms.

Fixed cost per SCSI command ≈ 1.9 ms (CBW, request/COMPLETE round trip, CSW).

Before the lane design: 12–14 MB/s.

### Raw hardware ceilings

No CPU in the data path (AUTO DMA channels, USB ↔ GPIF ↔ USB, both directions at once):

**188 MB/s each way, 61 GB, 0 data errors**

CPU-copy path (what the product uses):

**82–88 MB/s one way, D-cache off**

56 MB/s with the D-cache on (cache clean / invalidate per buffer); both CPUs 100 % busy, copy ≈ 130 µs per 16 KB buffer.

Tried and not adopted:
· zero-copy socket-to-socket MANUAL channel: mis-delivers back-to-back short buffers
· dedicated bulk lane (2nd GPIF thread pair, switched channels): ≈ 57 MB/s, no gain, the channel swap costs ≈ 3.3 ms per command.

### Design constraints

· 512 KB SRAM per board, so no sector cache; every sector is fetched from the PC.

· GPIF carries whole 16 KB buffers only; a short buffer waits for the next one, so every message occupies a full buffer (max payload 16 352 B, started 24 B into the buffer).

· A write is acknowledged to the OS only after the PC has written it to the image.

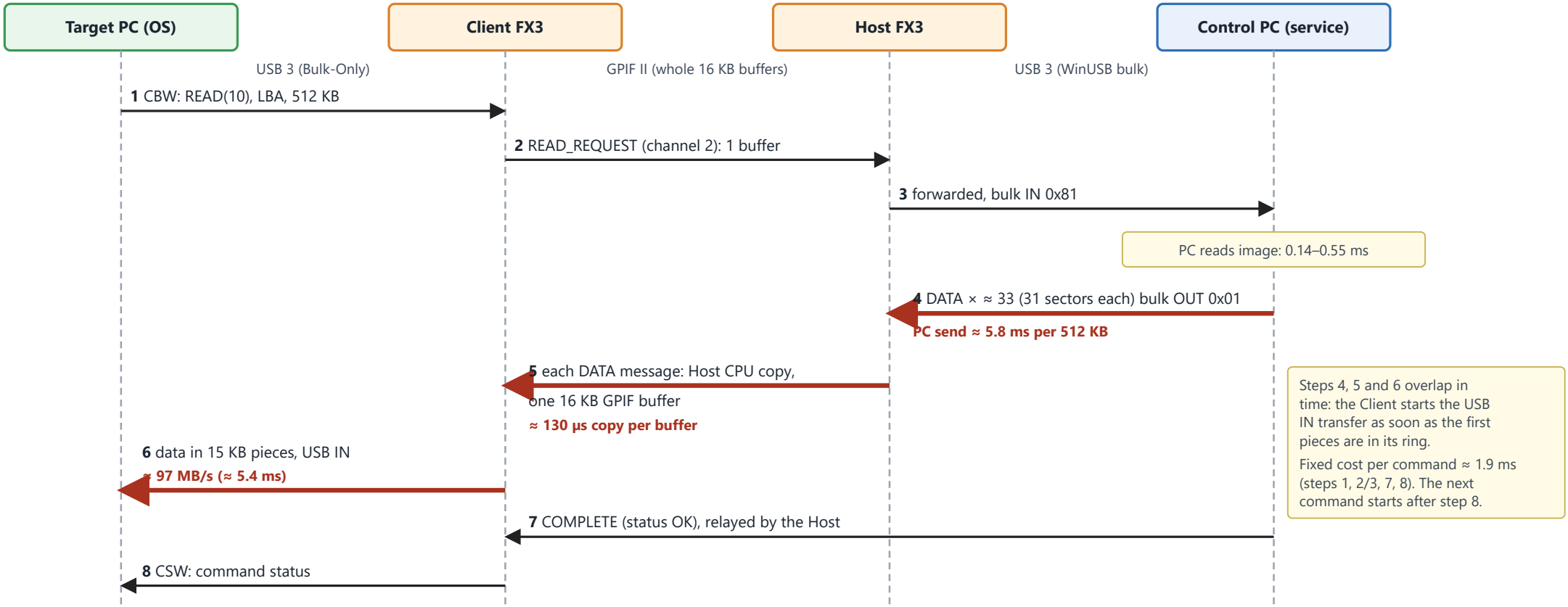
· No ARQ, no check word on the GPIF hop; USB protects its own hops with CRC-16.

· PIB DLL core 14 / sync 1 / output 14 and PCLK 100.8 MHz (403.2 MHz system clock / 4).

· The GPIF hop reports one PIB underrun per 16 KB buffer sent Client → Host (0xC0A) while the data stays bit-exact; cause not explained.

# Command flow and message layer

Life of one 512 KB sequential disk READ (what Windows issues for sequential access), the lane message format and the firmware threads




## One GPIF buffer = one message (16 384 bytes, not to scale)

| +0..23 unused | +24..31 8 B header | +32 … payload, up to 16 352 B (padded to 4 B) 32-byte aligned: USB DMA can fill / drain it directly | stale bytes never looked at |
|---|---|---|---|

Header: byte 0 channel · byte 1 type · bytes 2–3 tag (request id) · bytes 4–7 payload length. Whole buffer is always committed; a buffer without a plausible header (channel 0 / type 0, length too big) is dropped. No sequence numbers, no retransmission, no check word: flow control is the DMA buffer chain, the layers above use timeouts.

### Channels

0 control: HELLO, STATUS, PING/PONG, reset, update switch
1 CD-ROM: media state, TOC blobs, READ_REQUEST / DATA / COMPLETE
2 disk: attach, READ/WRITE/FLUSH/UNMAP requests, DATA, COMPLETE
3 keyboard + mouse reports, LED state
4 MIDI byte stream · 5 serial byte stream (credit windows)
6 Client firmware update · 7 Host firmware update

### Disk request path in one line

Target READ → Client READ_REQUEST → PC reads image → DATA → ring → USB
Target WRITE → Client WRITE_REQUEST + WRITE_DATA → PC writes image → COMPLETE → Client sends CSW. 60 s without an answer → medium error.

## Firmware threads and supervision

### Host board (ThreadX)

application thread (USB events, update-mode switch) · PC link thread · client manager + GPIF lane thread · MIDI/serial bridge thread · guard thread

### Client board (ThreadX)

CD-ROM thread · storage thread (SCSI, one command at a time) · GPIF lane thread + helper thread · peripheral worker (HID, MIDI, CDC) · power thread · guard thread. Worker threads sit below the guard so a spinning thread cannot starve the watchdog clear.

### Supervision

Host pings the Client every second; silent for 3.5 s → marked lost, Client reset over GPIO22 (never within 8 s of its last reset). PC absent: media is held, reads fail with a sector error after 60 s. Hardware watchdog 8 s.

### Boot and update

Both boards boot their own firmware from a local I2C EEPROM (1 MHz). Updates are LZSS-compressed containers signed with HMAC-SHA256 and are programmed through the Host's single WinUSB "Update Mode" device (both boards together).

Debug output is compiled out in the release firmware shown in these figures.